ScrollGuard Agent Benchmark

Phase 2 benchmark design report · Special Topics on Generative AI (Agentic AI) · 2026-11-01 (v3.1)
Repository: github.com/Blue-dogy/Advance_Topics_Generative-AI-ScrollGuard

Summary: answers to the assignment requirements

1. **Task the agent performs:** Given one smartphone-usage situation, the agent chooses `no_intervention`, `gentle_prompt` or `ask_context` and returns its reason as JSON. In track B, the agent **queries the touch log and user context itself with tools** before deciding.
2. **Evaluation data:** 81 cases (labels 32 / 31 / 18). Real usage, mood and typical-usage values from a public dataset (micro_datasetA, 5,777 rows) are combined with a user state and a five-part observation log. Adversarial, boundary, gesture-counterfactual and shortcut cases are included so that **no single number answers the case**.
3. **Success criteria and metrics:** accuracy (95% CI), macro-F1, detection rate, **false-intervention rate**, context-check rate, cost score, hard-case accuracy, counterfactual-pair metrics, run-to-run consistency and tool-use metrics. The agent is compared with six baselines to judge whether it does better *because it reads the context*.
4. **Evaluability:** inputs are separated from labels, scoring is automatic, every model uses the same prompt and harness, and leakage checks and inter-annotator agreement (kappa) are included. Each step is reproducible with one command.

1. Task definition

Project goal. Unlike lock apps that block by usage time alone, ScrollGuard **reads the usage context, leaves intentional use alone and steps in gently only during mindless scrolling (autopilot)**. The benchmark therefore measures the ability to decide whether to intervene, given the context.

1.1 Input and output

Item	Content
Input (15 columns)	**Situation:** time of day, entry trigger (self_open / notification / search), app type **User:** stated goal (or why it is blank), user state (who they are, what kind of day it is, where they are and what they are doing), self-reported mood (1-6), typical hourly usage **Observation log:** five parts: Entry / In app / App switches / Interactions (searches, saves, likes, comments) / Schedule and settings **Usage and gestures:** usage in the last 60 minutes, sessions, median session length, usage concentration, swipe-interval coefficient of variation (CV), average dwell per item
Output	One line of JSON `{"action": "...", "reason": "..."}`. Any action other than the three is scored as invalid.

1.2 Actions and labeling rules

Action (intent)	Rule
`no_intervention` (intentional)	Behavior matches the stated goal, or goal-directed actions are observed (searching, saving, writing, switching to a related app, paying, ...). Short utility use, or use within a self-set limit. **Long usage alone is not a reason to intervene.**
`gentle_prompt` (autopilot)	**Two or more** independent autopilot signals and no counter-evidence. Examples: no goal-directed action, drifting into a feed after a notification, passing a stated goal, timer or bedtime goal, a schedule conflict, repeated rechecking.
`ask_context` (undecided)	Purpose and drift signals conflict, or key information is inaccessible while usage is notable. Asking is better than a wrong intervention.

Boundary rules: explicit user declarations are respected. Exceeding a time limit alone is not enough evidence. Very short use (1-3 minutes) does not trigger a question even when information is missing.

1.3 Two evaluation tracks

Track	What the agent does	Script
A. One-shot decision	Receives all 15 inputs and decides once.	`run_agent.py`
B. Touch track (tool-using agent)	Starts with only time, entry trigger, app type and usage numbers, and gathers the rest by calling tools (up to 4 calls per case). `get_touch_log` recent swipe intervals (s), speed (px/s) and direction · `compute_stats` mean, CV and more · `get_user_context` stated goal, user state, mood and observation log The **trajectory** (what the agent checked before deciding) is logged and scored.	`run_agent_tools.py` `build_touch_logs.py`

1.4 Sensing-layer assumption

A real app has a **sensing layer** (collecting logs through usage-stats, notification, accessibility and calendar permissions) and a **decision layer** (the agent that chooses an action). This benchmark evaluates the **decision layer**. The observation log is assumed to be the correct output of the sensing layer; sensing accuracy, privacy cost and iOS restrictions are out of scope.

Input	How it would be collected (Android)
App, usage time, sessions, app switches	UsageStatsManager (usage-access permission)
Entry trigger	NotificationListenerService: whether the app was opened right after a notification
Swipe intervals, dwell, searches, saves, likes	AccessibilityService scroll and click events and screen names
Schedule, bedtime goal, timers, stated goal	Calendar permission, ScrollGuard settings and its goal question

1.5 Mapping to the pitch deck (Phase 1)

Pitch deck	How the benchmark reflects it
Context-Blind (lock apps cannot tell research from mindless swiping)	This distinction is the task itself. The time-threshold baseline (B1) quantifies the limitation with a 41% false-intervention rate, and adversarial cases are wrong if only time is used.
Gesture Tracking (scroll speed, pauses, direction)	Swipe-interval CV and average dwell (track A); interval, speed and direction sequences (track B, `get_touch_log`). Pauses appear as long intervals.
Zombie Detection (uniform, mechanical swiping)	Uniform swiping is a key signal, but the deck's claim is refined: some uniform swiping is intentional (binge-reading a webtoon, sorting photos). Twelve gesture-counterfactual cases test that gestures alone are wrong (B3: pair_both_correct 0.00).
Contextual Analysis (when, why and how apps are used)	Stated goal, user state, five-part observation log, typical usage (real values), schedule and bedtime goal
Silent Operation (no rigid lockouts)	The action space has no blocking. The default is `no_intervention`, and an unnecessary intervention has the highest cost (1.0).
Grayscale Fade · Gentle Friction · Restored Choice (pop-up or TTS)	`gentle_prompt` = grayscale fade with a small pop-up or TTS check-in; the user can always continue (stated in the agent prompt). `ask_context` = light friction that asks for intent first.
"Help users step away voluntarily, not by force"	The false-intervention rate is a core metric, and questions are also treated as interruptions with a cost (Section 3.2).

Classifying raw touch coordinates would be a time-series model evaluation, not an agent evaluation. Instead, the touch log is an **observation tool the agent queries when needed**, which gives the shape of an agentic benchmark from the lecture (tools, multiple turns, trajectory).

2. Test cases and evaluation data

2.1 Composition

Case type	Design intent	no_int.	gentle	ask	Total
standard	Typical cases	10	13	5	28
adversarial	Wrong if only usage time, sessions or concentration are used (e.g. 50 minutes of video for exam study)	6	3	0	9
boundary	Mixed signals or blocked information	2	1	13	16
counterfactual	Six pairs with the same context and usage but **opposite gestures**. The label follows the context, so both cases in a pair have the same label	6	6	0	12
shortcut	Cases that go **against single-cue shortcuts** such as dwell time, CV, stated-goal presence, app type, missing gestures, late night or text length	8	8	0	16
Total		**32**	**31**	**18**	**81**

2.2 Data sources and generation

- **Real data:** four usage numbers (usage time, sessions, median session length, usage concentration) come from real rows of the public dataset `micro_datasetA.csv` (5,777 rows × 17 columns), sampled with a fixed seed (42). The **self-reported mood (valence)** of the same row and the **median hourly usage** of the same user

over their other records (about 18 on average) are also real. Both intentional cases above typical usage and autopilot cases at typical usage exist, so comparing with typical usage alone does not decide the label.

- **Written data:** time of day, entry trigger, app type, swipe and dwell values, stated goals, user states (`user_states.py`) and observation logs were written to follow the labeling rules. The observation log records **only observed events and counts** (no summaries such as "no search"). Example: *Entry: tapped a recommended-video notification | In app: played 412 short videos in a row from the recommended feed | App switches: — | Interactions: searches 0, saves 0, likes 0, comments 0 | Schedule/settings: class tomorrow 09:00*. For the 49 cases without a stated goal, the reason (skipped, turned off, closed the window) is shown instead of a blank and is assigned independently of the label. The track-B touch log (8-40 swipes per case) has intervals, speeds and directions: intervals are drawn from a log-normal distribution to match each case's CV and mean dwell within ±10%, speeds are more uniform when intervals are uniform, and directions (0-15% backward swipes) are random and label-independent.
- **Language:** the official evaluation language is English (`*_en.csv`). Texts were translated from the Korean source; scripts check that numeric and categorical columns match the Korean files and that every number in a sentence matches the original.

2.3 Data quality control

- **Inputs separated from labels:** the agent receives only `benchmark_inputs_en.csv`. Labels (`benchmark_labels.csv`) and rationales and case types (`benchmark_meta*.csv`) are used only for scoring.
- **Leakage check:** `check_leakage*.py` searches the inputs for words that state the answer ("intentional", "autopilot", "addict", ...). Currently zero hits.
- **Consistency check:** `check_consistency.py` verifies that the observation log agrees with the numeric columns (entry trigger, repeat count = sessions, in-app minutes ≤ usage minutes, item count × mean dwell ≤ usage minutes). All 81 cases pass. This check found a case that "watched a 52-minute video to the end" with only 47.7 minutes of usage, which was fixed.
- **Label-rule audit:** all 81 cases were checked against the labeling rules. Three `gentle_prompt` cases (S002, S022, S044) had only one independent signal; a schedule conflict or a passed bedtime goal was added so they meet the two-signal rule.
- **Shortcut check:** we measured the best accuracy of rules that answer from a single number, category or word without understanding the context. In the first 45 cases a single CV threshold reached 0.76, which exposed a design flaw; gesture-counterfactual pairs and shortcut cases were added. Earlier context sentences also summarized the rule ("no search, no saves"), so "intervene if the text contains a negation" alone reached 0.60. The contexts were therefore rewritten as observation logs.

Single-cue rule	Earlier version*	Current (v3.1)
Swipe CV threshold (intervene if `CV < 0.6`)	**0.76**	0.49
Mean dwell threshold (best threshold)	**0.72**	0.53
Stated-goal presence, app type, late night, text length, ...	about 0.6	0.43-0.54
Intervene if the text contains a negation (no, not, without)	**0.60**	0.40
Mood, typical usage, usage-to-typical ratio (new columns)	-	0.44-0.49
B4 word classifier (trained on labels, diagnostic)	**0.72**	0.60
Majority class (always the most frequent label)	0.42	0.40

* CV, dwell and stated-goal rules were measured on the first 45 cases; the negation rule and B4 on the 81-case version before the observation-log rewrite.

- **Independent label validation:** two people who did not design the benchmark fill in `annotation_sheet_en.csv` without seeing the labels, and `agreement.py` computes Cohen's kappa. Kappa of 0.6 or higher supports label

validity; below 0.4 means the labeling rules must be redefined.

3. Success criteria and metrics

3.1 Metrics

Metric	Definition	Why it matters
accuracy (+95% CI)	Overall accuracy; CI from 2,000 bootstrap samples	Basic performance. With 81 cases the interval is wide, so differences need the CI
macro-F1, confusion matrix	Mean F1 over the three actions	Exposes bias toward one action
detect_rate	Share of `gentle_prompt` cases answered correctly	Does the agent catch autopilot use?
false_intervention	Share of `no_intervention` cases answered with `gentle_prompt`	**Core of the product philosophy.** Interrupting intentional use costs trust
question_burden	Share of `no_intervention` cases answered with `ask_context`	Unneeded questions also burden the user
ask_recall	Share of `ask_context` cases answered correctly	Ability to ask when unsure. All rule baselines score 0
cost_score	1 − mean error cost (cost table in 3.2)	Errors differ in harm
hard_acc	Accuracy on adversarial, boundary and counterfactual cases	Prevents easy cases from inflating the score
pair_both_correct, **gesture_flip**	Share of counterfactual pairs with both cases correct / share of pairs whose decision changed when only the gesture changed	Counterfactual evaluation: does the agent decide by context rather than gesture?
invalid_rate	Share of malformed or invalid outputs (scored as wrong)	Following the output format is part of performance
consistency	Share of cases with the same answer across 3 runs (temperature 0.3)	Reliability: being right once is not being right every time
Tool metrics (track B)	context_check_rate, touch_check_rate, mean_tool_calls, accuracy with and without checking context	Did the agent check the right evidence before deciding? (trajectory evaluation)

3.2 Error cost table (cost_score)

Label \ Prediction	no_intervention	gentle_prompt	ask_context
no_intervention	0	1.0 (unneeded intervention)	0.8 (unneeded question)
gentle_prompt	0.8 (miss)	0	0.5 (weaker response)
ask_context	0.6	0.6	0

Malformed or invalid outputs cost 1.0. **A question (ask_context) is also an interruption**, so asking during intentional use (0.8) costs nearly as much as intervening (1.0). With the first weights (unneeded question 0.5, asking on autopilot 0.4), a rule that always asks had the best cost score (0.65); this loophole was fixed, and

that rule now scores 0.49, fifth of the six baselines. Because the weights are design assumptions, `evaluate.py --sensitivity` reports cost scores and ranks under four weight sets (current, previous, miss-heavy, all errors equal). Among the baselines, B2 ranks first or second in all four sets.

3.3 Success criteria

An agent meets the project goal when it satisfies all of the following **together**. Using several criteria avoids optimizing one number at the expense of the goal (Goodhart's law).

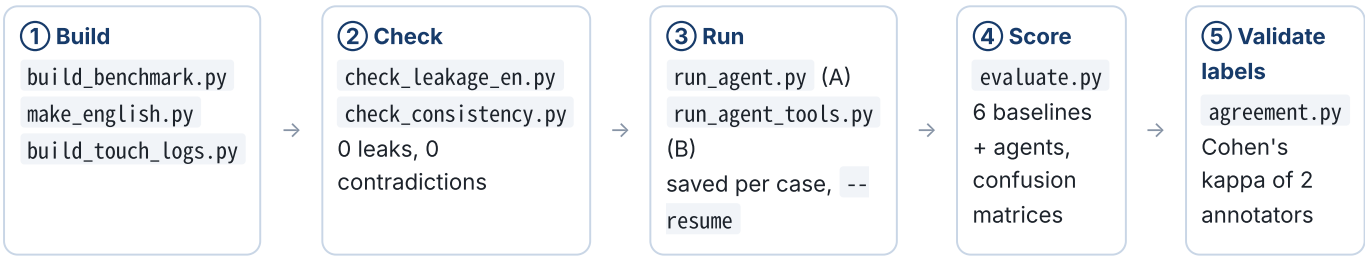
1. Accuracy and cost_score above the best baseline (B2: 0.51 / 0.64)
2. Low false_intervention (clearly below the time-threshold baseline B1's 0.41; target 0.10 or lower)
3. ask_recall clearly above 0 (all rule baselines are 0)
4. pair_both_correct above B3 (0.00) and a low gesture_flip (B3: 1.00)
5. Low invalid_rate and high consistency
6. Track B: a high context-check rate, and higher accuracy when the context was checked than when it was not

3.4 Baseline results (English inputs, 81 cases)

Baseline	accuracy	95% CI	detect	false_int.	ask_recall	hard_acc	pair_both	flip	cost
B0 always no_intervention	0.40	[0.30, 0.49]	0.00	0.00	0.00	0.38	0.50	0.00	0.56
B0 always gentle_prompt	0.38	[0.28, 0.49]	1.00	1.00	0.00	0.27	0.50	0.00	0.47
B0 always ask_context	0.22	[0.14, 0.32]	0.00	0.00	1.00	0.35	0.00	0.00	0.49
B1 intervene at 30+ minutes	0.41	[0.30, 0.52]	0.45	0.41	0.00	0.22	0.33	0.00	0.54
B2 feature rule, no text	**0.51**	[0.41, 0.60]	0.35	0.06	0.00	0.57	0.50	0.50	**0.64**
B3 swipe CV < 0.6	0.49	[0.40, 0.60]	0.65	0.38	0.00	0.46	0.00	1.00	0.61
B4 word classifier (diagnostic)	0.60	[0.51, 0.70]	1.00	0.44	0.17	0.65	1.00	0.00	0.69

Interpretation. The usage-time rule (B1) interrupts 41% of intentional use, which quantifies the limitation of time-only approaches (context blindness). No rule exceeds 0.51, so an agent clearly above it is performing better because it reads the context. B4 is a word classifier **trained on the labels** (leave-one-out), so it is not a competitor but a reference for how much label information the words carry. The remaining discriminative words carry meaning, such as feed and scrolled in a row (passive consumption) or searched and notes (goal-directed action); surface cues such as negations, text length and template phrases were removed. Results of real LLM agents (EXAONE 3.5 7.8B, Claude) will be added as rows of the same table.

4. How to evaluate an agent with the benchmark




```
python run_agent.py --lang en --inputs benchmark_inputs_en.csv --model <model> --runs 3 --out predictions_A.csv
python run_agent_tools.py --lang en --inputs benchmark_inputs_en.csv --model <model> --runs 3 --out predictions_B.csv
python evaluate.py --inputs benchmark_inputs_en.csv --meta benchmark_meta_en.csv --pred predictions_A.csv predictions_B.csv --confusion --sensitivity
```

Any team's agent can be scored on the same terms by submitting a prediction file in the same format (`case_id, run, action, reason`). Track B also keeps the `tools` column and the full conversation (`.jsonl`) for error analysis.

5. Mapping to the lecture (Evaluating LLMs)

Lecture topic	How the benchmark reflects it
Close-ended tasks allow automatic evaluation (classification)	Defined as choosing one of three actions and compared with labels automatically. The reason field is used only for error analysis
Loss does not reflect task performance	Metrics designed for the task goal: false-intervention rate, detection rate, per-error cost
Results depend on implementation (prompt, generation)	Same system prompt, JSON format, temperature and tool protocol for every model; format violations are invalid
Contamination; keep a private test set	Labels and meta files are separate from inputs. In Phase 3, other teams receive only the inputs and format; labels stay private
Goodhart's law; chasing numbers	Several metrics together form the success criteria (3.3). The cost score was checked against an "always ask" strategy and is reported with a sensitivity analysis (3.2)
Human evaluation: inter-annotator disagreement	Label validity checked with Cohen's kappa of two independent annotators
Agentic benchmarks score trajectories, tools and environments, not one answer	Track B: multi-turn interaction to gather evidence with tools, a tool budget (4 calls), trajectory logs and tool-use metrics
pass@k, pass^k, repeated trials and 95% CI	3 runs per case for consistency; bootstrap 95% CI for accuracy
Reward hacking and weak verifiers	Shortcut analysis finds and closes ways to answer without context; empty or malformed answers never count as success
A score reflects model + harness + budget	One fixed harness, only the model changes; model, temperature, runs and tool limit are reported

6. Status and limitations

Not done yet

- Real LLM agent results: inputs changed (observation logs, user state), so earlier EXAONE results are no longer valid, and results for the current version (tracks A and B, EXAONE and Claude) are not available yet. The table above contains baselines only.
- Independent labeling by two annotators and the kappa calculation

1. **Label validity:** the labels are the designers' judgment. Validity cannot be claimed until kappa is measured.
2. **Synthetic signals:** only the four usage numbers, mood and typical usage are real; swipes, dwell, stated goals, user states, observation logs and touch sequences are written or generated. The data does not capture real users' intervention preferences, and the sensing layer is assumed to be perfect (1.4).
3. **Sample size:** with 81 cases, the accuracy CI is about ±0.1. Differences smaller than 0.1 are not interpreted.
4. **Cost weights:** error costs are design assumptions. The "always ask" loophole was fixed, but under the miss-heavy weights that rule rises to third again. The cost score is therefore reported together with question_burden, ask_recall and the sensitivity analysis.
5. **Translation:** English texts were translated from Korean and may differ in subtle nuance.
6. **Scope of the leakage check:** keyword checks cannot catch implicit cues such as style. Single-cue rules and the B4 word classifier add coverage, but shortcuts that combine several cues cannot be fully ruled out.